

HANOI UNIVERSITY OF SCIENCE AND
TECHNOLOGY

SCHOOL OF INFORMATION AND COMMUNICATIONS
TECHNOLOGY



Solving a Multi-Team Task Scheduling Problem with Cost and Precedence Constraints

Author: Le Van Tuan Anh

Student ID: 202416657

Supervisor: Bui Quoc Trung

Date of Submission: June 20, 2025

Abstract

This report addresses the problem of assigning and scheduling tasks in a multi-team project environment, considering team availability, task durations, execution costs, and precedence constraints. The objective is to maximize the number of scheduled tasks, minimize the total completion time, and minimize the assignment cost, in the priority order. I propose multiple approaches, each designed for a specific purpose, with varying computational costs and differing levels of efficiency under real-world constraints..

Contents

1	Introduction	3
2	Problem Definition	4
3	Exact Methods	5
3.1	Integer Programming/Constraint Programming	5
3.2	Backtracking	6
3.2.1	Main idea	6
3.2.2	Pseudo Code	6
4	Heuristic	8
4.1	Greedy Algorithm	8
4.1.1	Main idea	8
4.1.2	Pseudo Code	8
4.2	Simple/Steepest-Ascent Hill Climbing	9
5	Meta-heuristic	10
5.1	Genetic Algorithm	10
5.1.1	Chromosome Representation	10
5.1.2	Population Initialization	11
5.1.3	Decode Chromosome and Fitness Function	12
5.1.4	Selection Method	14
5.1.5	Crossover Operator	14
5.1.6	Mutation	17
5.1.7	Evaluate and Replace	18
5.1.8	Termination Condition	19
5.1.9	Main Loop	19
5.2	Simulated Annealing	20
5.2.1	State Representation	20
5.2.2	Create Initial Feasible Solution	20
5.2.3	Decode State and Fitness Function	22
5.2.4	Neighbor Generation	23
5.2.5	Acceptance Probability	23
5.2.6	Termination Condition	23
5.2.7	Cooling Schedule and Main Loop	24
5.3	Tabu Search	24
5.3.1	State and Move Representation	25
5.3.2	Create Initial Feasible State	25
5.3.3	Decode State and Fitness Calculation	25

5.3.4	Neighbor Generation	25
5.3.5	Move Identification and Tabu Check	26
5.3.6	Termination Condition	27
5.3.7	Main Loop	27
6	Results and Conclusion	29
6.1	Results	29
6.2	Conclusion	30

Chapter 1

Introduction

Task scheduling and assignment are critical components of effective project management. In real-world scenarios, projects often consist of multiple tasks that must be completed by different teams with varying availability and capabilities. Moreover, constraints such as task dependencies (precedence constraints) and assignment costs make the scheduling problem more complex.

This report focuses on a scheduling problem where a project contains M tasks and N teams. Each team is available at a specific time, and each task can be performed by certain teams with a known cost. Additionally, some tasks must be completed before others can begin. The goal is to determine an optimal schedule that assigns each task to a team and specifies its start time, while optimizing the following objectives in order:

1. Maximize the number of scheduled tasks.
2. Minimize the total completion time.
3. Minimize the overall assignment cost.

Chapter 2

Problem Definition

Let the project consist of M tasks, indexed by $i = 1, 2, \dots, M$, and N teams, indexed by $j = 1, 2, \dots, N$.

- $d(i)$: duration of task i
- $s(j)$: available starting time of team j
- $c(i, j)$: cost for team j to perform task i
- $Q = \{(i, j)\}$: precedence constraints, meaning task j can start only after task i is completed

The objective is to determine:

- A mapping of each task i to a team j that can perform it
- A start time for each task

Subject to the following constraints:

1. Precedence constraints: if $(i, j) \in Q$, then task j must start after task i ends
2. Team availability: task i cannot be assigned to team j before $s(j)$
3. Each team can only perform one task at a time

The scheduling must prioritize:

1. Maximizing the number of tasks scheduled
2. Minimizing the total completion time (i.e., latest finishing time)
3. Minimizing the total cost

Chapter 3

Exact Methods

3.1 Integer Programming/Constraint Programming

Decision Variables and Constraints

Decision Variables

- $x_{i,j}$: Binary variable; 1 if assigned task i to team j
- $start_i$: Integer variable representing the begin time of task i
- z_i : Binary variable; 1 if task i is scheduled
- $completion$: Integer variable indicating overall completion time

Constraints

1. **Team assignment only if task is scheduled**

$$\sum_{j=1}^N x_{i,j} = z_i, \quad \forall i = 1, \dots, M$$

2. **Team must available before doing task**

$$start_i \geq s_j * x_{i,j} \quad \forall i = 1, \dots, M; j = 1, \dots, N$$

3. **Precedence constraints**

$$start_j \geq start_i + d(i) * (bigM * (1 - z_i) + z_i) \quad \forall (i, j) \in Q$$

4. **No team can execute two tasks simultaneously**

$$start_i + d(i) \leq start_j + bigM * (2 - x_{i,k} - x_{j,k})$$

OR

$$start_j + d(j) \leq start_i + bigM * (2 - x_{i,k} - x_{j,k})$$

$$\forall i, j = 1, \dots, M; k = 1, \dots, N$$

5. **Completion time**

$$completion \geq start_i + d(i) \quad \forall i = 1, \dots, M$$

Objectives

1. Maximize number of scheduled task

$$\max \sum_{i=1}^M z_i$$

2. Minimize completion time

$$\min \text{completion}$$

3. Minimize cost

$$\min \sum_{\substack{i=1, \dots, M \\ j=1, \dots, N}} x_{i,j} * c(i, j)$$

3.2 Backtracking

Introduction: Backtracking optimizes the objectives by exploring all the possible combinations through a DFS-like method. This ensures the optimality of the returning result.

3.2.1 Main idea

Since the algorithm explore all the feasible results and eliminates non-optimal solutions, the problem of scheduling and assignment with M tasks, N teams and Q precedences can be solved as follows:

- **Initialization:** Start with an empty schedule
- **Construction:** Iteratively construct the schedule with a topo-sorted list of possible tasks at the time of iteration and the current schedule
- **Optimality Check:** If the current schedule is showing non-optimality compared to the current best result, eliminates and backtrack. Else, update the current best schedule.
- **Return the best result:** After finish exploring, return the best schedule found.

3.2.2 Pseudo Code

Helper function: Calculate Score

```
1: function CALC_SCORE(schedule: List[List[task: int, team: int, start_time: int]])
2:   schedule ← [schedule[i] for i in range M if schedule[i] ≠ [-1, -1, -1]]
3:   countTask ← length(schedule)
4:   completionTime ← -1
5:   totalCost ← 0
6:   for task, team, start_time in schedule do
7:     totalCost ← totalCost + c[task, team]
8:     if start_time + duration[task] > completionTime then
9:       completionTime ← start_time + duration[task]
10:    end if
```

```

11:   end for
12:   return countTask, completionTime, totalCost
13: end function

```

Algorithm 1: Backtracking Approach

```

1: Initialize:
2:   best_schedule  $\leftarrow$  None
3:   best_count, best_time, best_cost  $\leftarrow$  -1,  $\infty$ ,  $\infty$ 
4:   currentSchedule  $\leftarrow$   $[-1, -1, -1] * M$ 
5:   taskStart  $\leftarrow$   $[0] * M$ 
6:   taskEnd  $\leftarrow$   $[\text{None}] * M$ 
7:   taskAvailable  $\leftarrow$  List of tasks with no precedences
8:   function ASSIGN(currentOrder, taskAvailable)
9:     if taskAvailable is empty then:
10:       currentCount, currentTime, currentCost  $\leftarrow$  Calc_Score(currentSchedule)
11:       if [currentCount, currentTime, currentCost] > [best_count, best_time,
12:         best_cost] then
13:         best_schedule  $\leftarrow$  currentSchedule
14:         best_count, best_time, best_cost  $\leftarrow$  currentCount, currentTime, current-
15:           Cost
16:       end if
17:     end if
18:     for currentTask in taskAvailable do
19:       for team in teams can do currentTask do
20:         earliest_time  $\leftarrow$  max(s(team), max(taskEnd[predecessors of current-
21:           Task]))
22:         nextAvailable  $\leftarrow$  set of successors of currentTask
23:         currentSchedule[currentOrder]  $\leftarrow$  [currentTask, team, earliest_time]
24:         s(team)  $\leftarrow$  earliest_time + d(currentTask)
25:         taskStart[currentTask]  $\leftarrow$  earliest_time
26:         taskEnd[currentTask]  $\leftarrow$  earliest_time + d(currentTask)
27:         assign(currentOrder + 1, taskAvailable + nextAvailable -
28:           set(currentTask))
29:         revert currentSchedule, s, taskStart, taskEnd
30:       end for
31:     end for
32:   end function
33: Call: assign(0, taskAvailable)

```

Chapter 4

Heuristic

4.1 Greedy Algorithm

Introduction: Exact methods can guarantee optimal results but are computationally expensive. Greedy algorithms offer a faster alternative by making step-by-step decisions that reduce time complexity while still producing reasonably good schedules in practice.

4.1.1 Main idea

In contrast of exploring through all the feasible combinations, greedy approach builds the solution step by step by always choosing the locally best options. While it doesn't guarantee a global optimal result, it drastically reduces computation time and often yields effective, reasonable sub-optimal schedules.

1. **Initialization:** Start with an empty schedule
2. **Construction:** At every step, take out a task from a topo-sorted list of tasks
3. **Greedy Selection:** Choose the team that can do currently selected task that introduce the best (time, cost) combination
4. **Return the result:** Return the constructed schedule

4.1.2 Pseudo Code

Algorithm 2: Greedy Algorithm

```
1: Initialize:  
2:    $taskStart \leftarrow [0] * M$   
3:    $taskEnd \leftarrow [None] * M$   
4:    $order \leftarrow$  topo-sorted set of  $tasks$   
5:    $schedule \leftarrow []$   
6: function GREEDY_ALGORITHM  
7:   for currentTask in order do  
8:      $best\_choice \leftarrow None$   
9:      $best\_time \leftarrow \infty$   
10:     $best\_cost \leftarrow \infty$   
11:    for team in teams can do currentTask do
```

```

12:          $earliest\_time \leftarrow \max(s(team), \max(taskEnd[predecessors\ of\ current-$ 
Task]))
13:          $cost \leftarrow c(currentTask, team)$ 
14:         if ( $earliest\_time < best\_time$ ) or ( $earliest\_time == best\_time$  and  $cost$ 
 $< best\_cost$ ) then
15:              $best\_time \leftarrow earliest\_time$ 
16:              $best\_choice \leftarrow team$ 
17:              $best\_cost \leftarrow cost$ 
18:         end if
19:     end for
20:     if  $best\_choice \neq None$  then
21:          $schedule\ append\ [currentTask, best\_choice, best\_time]$ 
22:          $taskStart[currentTask] \leftarrow best\_time$ 
23:          $taskEnd[currentTask] \leftarrow best\_time + d(currentTask)$ 
24:          $s(best\_choice) \leftarrow best\_time + d(currentTask)$ 
25:     end if
26: end for
27:     return  $schedule$ 
28: end function

```

4.2 Simple/Steepest-Ascent Hill Climbing

A detailed discussion of Simulated Annealing and Tabu Search—both of which are enhancements of the Hill Climbing algorithm—will be presented in the later chapters. As such, including them here would be redundant.

Chapter 5

Meta-heuristic

5.1 Genetic Algorithm

Introduction: Genetic Algorithm (GA) are a type of evolutionary algorithm inspired by natural selection and genetics. They solve complex optimization and search problems by evolving a population of candidate solutions through successive generations, gradually improving their quality.

Main idea: By repeatedly selecting the fittest individuals, combining their traits, and introducing random mutations, GA balances exploration (searching broadly) and exploitation (refining good solutions) to find high-quality solutions over time.

1. **Initialization:** Start with a population of random candidate solutions (called chromosome)
2. **Selection:** Choose the best candidates based on a *fitness* function
3. **Crossover:** Combine parts of two parents to create new offspring
4. **Mutation:** Randomly alter some genes to maintain diversity
5. **Replacement:** Form a new generation, and repeat the process.

5.1.1 Chromosome Representation

In GA implementation, each individual in the population is represented by a chromosome, which is made up by two lists: *task_order* and *team_assignment*.

- **Structure:** chromosome: {
 "task_order" : [3, 1, 4, 2, 5],
 "team_assignment" : [2, 1, 3, 1, 2]
}
- **Task Order:** "*task_order*" represent a topologically valid order of task.
- **Team Assignment:** "*team_assignment*" indicate a randomly assigned list of teams that are capable of doing each task.

Example: A chromosome: {"task_order": [3, 1, 2], "team_assignment": [2, 1, 2]} shows that:

- Task are being done in the order of $3 \rightarrow 1 \rightarrow 2$
- Task 3, 2 is done by team 2; task 1 is done by team 1

5.1.2 Population Initialization

The first step when running GA. This step sets up the initial population of solutions (chromosomes), each representing a schedule of tasks assigned to teams with respect to constraints. This step runs as follows:

1. Generate task order (topological sort)
2. Assign teams randomly
3. Combine into an individual
4. Initialize the whole population

Pseudo Code

Helper Function: Random Topological Sort

```

1: function RANDOM_TOPOLOGICAL_SORT(num_task, precedence, can_do)
2:   in_deg  $\leftarrow$  [0] * num_task
3:   precedence_graph  $\leftarrow$  {u: [] for u in range(num_task)}
4:   for (u, v) in precedence do
5:     precedence_graph[u] append v
6:     in_deg[v]  $\leftarrow$  in_deg[v] + 1
7:   end for
8:   line  $\leftarrow$  queue([u for u in range(num_task) if in_deg[u] == 0])
9:   order  $\leftarrow$  []
10:  while line is not empty do
11:    shuffle line
12:    u  $\leftarrow$  line.pop()
13:    order append u
14:    for v in precedence_graph[u] do
15:      in_deg[v]  $\leftarrow$  in_deg[v] - 1
16:      if in_deg[v] == 0 then
17:        line push v
18:      end if
19:    end for
20:  end while
21:  return order
22: end function

```

Helper Function: Construct Chromosome

```

1: function CONSTRUCT_CHROMOSOME(num_task, precedence, available_team,
   can_do)
2:   task_order  $\leftarrow$  Random_Topological_Sort(num_task, precedence, can_do)
3:   team_assignment  $\leftarrow$  [random_choice(available_team[i]) for i in task_order]

```

```

4:   chromosome ← {"task_order": task_order, "team_assignment":
   team_assignment}
5:   return chromosome
6: end function

```

Operation: Initialize Population

```

1: Initialize:
2:   available_team ← dictionary of teams can do each task
3:   can_do ← binary list, 1 if the task can be done by some teams, 0 otherwise
4:   function INITIALIZE_POPULATION(population_size, num_task, precedence,
   available_team, can_do)
5:     population ← []
6:     for i ← 1 to population_size do
7:       individual ← Construct_Chromosome(num_task, precedence, avail-
   able_team, can_do)
8:       population append individual
9:     end for
10:    return population
11: end function

```

5.1.3 Decode Chromosome and Fitness Function

Since the problem is multi-objective, **lexicographic fitness** is used:

```

fitness = (
    num_unscheduled_task,
    completion_time,
    total_cost
)

```

The fitness function evaluates the quality of each chromosome by calculating each criterion. And based on the defined fitness, there are three main metrics:

- **Number unscheduled tasks:** The number of unscheduled task, aiming to maximizing the total number of scheduled tasks
- **Completion time:** Overall completion time, aiming to minimize the total time cost
- **Assigning cost:** Overall assignment cost, aim to minimize the total cost

1. Simulate task execution in **task_order**

2. For each task:

- Find **earliest time** after:
 - Team's availability $s(j)$
 - Completion time of all predecessors
 - Finish of previous task by the same team
- If feasible, schedule it and compute

- Start time
- End time
- Cost $c(i, j)$

3. Finally, keep track of:

- Total number of task scheduled
- Completion time
- Total cost

Pseudo Code

Helper Function: Decode Chromosome

```

1: function DECODE_CHROMOSOME(chromosome, duration)
2:   schedule  $\leftarrow []$ 
3:   taskStart  $\leftarrow [\text{None}] * M$ 
4:   taskEnd  $\leftarrow [0] * M$ 
5:   teamAvailable  $\leftarrow s$ 
6:
7:   task_order, team_assignment  $\leftarrow$  chromosome
8:   queue_length  $\leftarrow$  length(task_order)
9:   for idx in range queue_length do
10:    currentTask, team  $\leftarrow$  task_order[idx], team_assignment[idx]
11:    earliest  $\leftarrow \max(\text{teamAvailable}[\text{team}], \max(\text{taskEnd}[\text{predecessors of currentTask}]))$ 
12:    teamAvailable[team]  $\leftarrow$  earliest + duration(currentTask)
13:    taskStart[currentTask]  $\leftarrow$  earliest
14:    taskEnd[currentTask]  $\leftarrow$  earliest + duration(currentTask)
15:    schedule append (currentTask, team, earliest)
16:  end for
17:  return schedule
18: end function

```

Helper Function: Calculate Fitness Score

```

1: function CALC_FITNESS(chromosome, cost, duration)
2:   schedule  $\leftarrow$  Decode_Chromosome(chromosome, duration)
3:   numTask, completion, totalCost  $\leftarrow$  0, 0, 0
4:   for task, team, start_time in schedule do
5:     numTask  $\leftarrow$  numTask + 1
6:     completion  $\leftarrow$  max(completion, start_time + duration(task))
7:     totalCost  $\leftarrow$  totalCost + cost(task, team)
8:   end for
9:   return (M - numTask, completion, totalCost)
10: end function

```

5.1.4 Selection Method

The GA employs **Rank Selection** to choose parent chromosomes for reproduction. This involves the following steps:

1. **Sorting:** Arrange the population in ascending order based on their fitness score
2. **Probability Assignment:** Assign a selection probability to each individual based on their rank, with higher-ranked individuals having a higher probabilities. Scaling factor $sp = 1.5$ is used to adjust the probabilities.
3. **Parent Selection:** Select two parents probabilistically based on the assigned probabilities in order to generate next generation.

Pseudo Code

Operation: Parent Selection by Rank

```
1: function RANK_SELECTION(population)
2:   sort population based on fitness_score
3:    $k \leftarrow \text{length}(\text{population})$ 
4:   Initialize probs  $\leftarrow [0] * k$ 
5:   parents  $\leftarrow []$ 
6:    $sp \leftarrow 1.5$ 
7:   for  $i \leftarrow 1$  to  $k$  do
8:     probs[i]  $\leftarrow \frac{1}{k} \times (sp - \frac{(2 \times sp - 2) \times (i - 1)}{k - 1})$ 
9:   end for
10:  for  $i \leftarrow 2$  to  $k$  do
11:    probs[i]  $\leftarrow \text{probs}[i] + \text{probs}[i - 1]$ 
12:  end for
13:  for 2 iterations do
14:     $r \leftarrow$  random number between 0 and 1
15:    for  $i \leftarrow 1$  to  $k$  do
16:      if  $r \leq \text{probs}[i]$  then
17:        parents append population[i]
18:        break
19:      end if
20:    end for
21:  end for
22:  return parents
23: end function
```

5.1.5 Crossover Operator

The crossover operator is a key genetic algorithm mechanism that combines parts of two parent solutions to create new offspring. Inspired by biological reproduction, crossover aims to exploit good traits from both parents and pass them to the next generation.

- Take two parent chromosomes
- Mix their information to produce two more children

- Children inherit the strengths of both parents, increasing the chance of better solutions

We care about 2 main crossover:

1. Precedence-preserving Order-based Crossover (POX)
2. Crossover for Team Assignment

Precedence-preserving Order-based Crossover

This is a special crossover that respects task dependencies.

Steps:

1. Start with an empty child list: `child = [-1] * length(parent1)`
2. Choose a random subset of *parent1* and copy them into child at the exact same positions
3. Fill the remaining positions with tasks from *parent2* in order, skipping already added ones
4. Check if precedence is violated. If is, do a topo-sort to restore a valid order.

Pseudo Code

Helper Function: Precedence Feasible Check

```

1: function IS_PRECEDENCE_FEASIBLE(task_order, precedence)
2:   pos  $\leftarrow$  {task: idx for idx, task in enumerate(task_order)}
3:   for (i, j) in precedence do
4:     if pos[i]  $\geq$  pos[j] then
5:       return False
6:     end if
7:   end for
8:   return True
9: end function

```

Operation: Precedence-preserving Order-based Crossover

```

1: function POX(parent1, parent2, precedence)
2:   child  $\leftarrow$  [-1] * length(parent1)
3:   for i  $\leftarrow$  1 to length(parent1) do
4:     r  $\leftarrow$  random number between 0 and 1
5:     if r  $\geq$  0.5 then
6:       child[i]  $\leftarrow$  parent1[i]
7:     end if
8:   end for
9:   j  $\leftarrow$  1
10:  for i  $\leftarrow$  1 to length(child) do
11:    if child[i] == -1 then
12:      while parent2[j] in added do
13:        j  $\leftarrow$  j + 1

```

```

14:         end while
15:         child[i] ← parent2[j]
16:         j ← j + 1
17:     end if
18: end for
19: if not Is_Precedence_Feasible(child, precedence) then
20:     child ← topo_sort(child)
21: end if
22: return child
23: end function

```

Team Assignment Crossover

The goal of **Team Assignment Crossover** is to:

- Inherit good cost-efficient or feasible team assignments from both parents
- Create diversity in the population
- Potentially reduce total cost or improve task feasibility

At each step, randomly choose the team assignment from one of the two parents.

$child[i] \leftarrow random_choice([parent1_team[i], parent2_team[i]])$

Pseudo Code

Operation: Team Assignment Crossover

```

1: function TAX(parent1, parent2, available_team, task_order)
2:   child ← [-1] * length(parent1)
3:   for i ← 1 to length(parent1) do
4:     currentTask ← task_order[i]
5:     possible ← [lst[i] for lst in [parent1, parent2] if lst[i] in avail-
       able_team[currentTask]]
6:     if length(possible) > 0 then
7:       child[i] ← random_choice(possible)
8:     else
9:       child[i] ← random_choice(available_team[currentTask])
10:    end if
11:  end for
12:  return child
13: end function

```

Offsprings Generation

In order to get the new generations, we get the offsprings from the two above crossovers, and combine to get the two offsprings

Pseudo Code

Operation: Offspring Generation

```

1: function OFFSPRING_GENERATE(parent1, parent2, precedence, available_team)
2:   task_order1, team1 ← parent1

```

```

3:   task_order2, team2 ← parent2
4:
5:   child_task1 ← POX(task_order1, task_order2, precedence)
6:   child_team1 ← TAX(team1, team2, available_team)
7:
8:   child_task2 ← POX(task_order2, task_order1, precedence)
9:   child_team2 ← TAX(team2, team1, available_team)
10:
11:  child1 ← {"task_order": child_task1, "team_assignment": child_team1}
12:  child2 ← {"task_order": child_task2, "team_assignment": child_team2}
13:  return child1, child2
14: end function

```

5.1.6 Mutation

Mutation introduces small, random changes to the offsprings. For example:

- Changing the team assigned to a task
- Swapping two task in the order (if valid)

This improve the search in many ways:

1. Maintains Genetic Diversity
2. Avoids Local Optima
3. Explores the Search Space More Thoroughly
4. Improves Robustness

Two main mutations

- **Task Order Mutation:** Swap task order with constraints check
- **Team Assignment Mutation:** Randomly reassign task to another valid team

Pseudo Code

Helper Function: Task Order Mutation

```

1: function TASK_ORDER_MUTATION(task_order, precedence, attempts)
2:   for k ← 1 to attempts do
3:     new_order ← task_order
4:     i, j ← 2 random tasks from new_order
5:     swap new_order[i], new_order[j]
6:     if Is.Precedence.Feasible(new_order, precedence) then
7:       return new_order
8:     end if
9:   end for
10:  return task_order
11: end function

```

Helper Function: Team Assignment Mutation

```
1: function TEAM_ASSIGNMENT_MUTATION(team_assignment, task_order, avail-
   able_team, mutation_rate)
2:   new_assignment  $\leftarrow$  team_assignment
3:   for idx, task in enumerate(task_order) do
4:     r  $\leftarrow$  random number between 0 and 1
5:     if r  $\geq$  mutation_rate then
6:       viable  $\leftarrow$  available_team[task]
7:       new_assignment[idx]  $\leftarrow$  random_choice(viable)
8:     end if
9:   end for
10:  return new_assignment
11: end function
```

Operation: Full Mutation Function

```
1: function MUTATE(chromosome, precedence, available_team, attempts, muta-
   tion_rate)
2:   task_order, team_assignment  $\leftarrow$  chromosome
3:   new_task_order  $\leftarrow$  Task_Order_Mutation(task_order, precedence, attempts)
4:   new_team_assignment  $\leftarrow$  Team_Assignment_Mutation(team_assignment,
   new_task_order, available_team, mutation_rate)
5:   return {"task_order": new_task_order,
6:         "team_assignment": new_team_assignment}
7: end function
```

5.1.7 Evaluate and Replace

This is when we evaluate the new offsprings and replace them into the polulation. This step flow is as follows:

1. Append the offsprings to the current population
2. Evaluate and sort the population
3. Only keeping the first "population.size" of the population

Pseudo Code

Operation: Evaluate and Replace Function

```
1: function EVALUATE_AND_REPLACE(population, child1, child2, cost, duration)
2:   population_size  $\leftarrow$  length(population)
3:   population append child1, child2
4:   fitness_score  $\leftarrow$  [Calc_Fitness_Score(ind, cost, duration) for ind in population]
5:   sort population based on fitness_score
6:   return population[:population_size]
7: end function
```

5.1.8 Termination Condition

The last crucial piece of GA. Termination Condition tells the algorithm when to stop running and return the best solution found so far. In this problem, I use mainly three of these condition:

- **Maximum number of Generation:** Stop the algorithm after a number of generations (e.g. 500).
- **Convergence Criteria:** If there is no better solutions found after a number of consecutive generations (e.g. 50), early stopping the algorithm to conserve computational resource.
- **Maximum Time Limit:** Stop the algorithm after a pre-defined amount of time (e.g. 300 seconds).

5.1.9 Main Loop

Pseudo Code

Operation: Main Function

```
1: function MAIN
2:   start  $\leftarrow$  time.time()
3:   max_generations  $\leftarrow$  500
4:   max_time  $\leftarrow$  300
5:   max_converge  $\leftarrow$  50
6:   population  $\leftarrow$  Initialize_Population(100, num_task, precedence, available_team, can_do)
7:   best_individual  $\leftarrow$  population[0]
8:   best_fitness  $\leftarrow$  Calc_Fitness(best_individual, cost, duration)
9:   for gen  $\leftarrow$  1 to max_generations do
10:    current_time  $\leftarrow$  time.time()
11:    if current_time - start  $\geq$  max_time then
12:      break
13:    end if
14:    parent1, parent2  $\leftarrow$  Rank_Selection(population)
15:    child1, child2  $\leftarrow$  Offspring_Generation(parent1, parent2, precedence, available_team)
16:    child1  $\leftarrow$  Mutate(child1, precedence, available_team, 5, 0.15)
17:    child2  $\leftarrow$  Mutate(child2, precedence, available_team, 5, 0.15)
18:    population  $\leftarrow$  Evaluate_and_Replace(population, child1, child2, cost, duration)
19:    if gen % max_converge == 0 then
20:      current_individual  $\leftarrow$  population[0]
21:      current_best  $\leftarrow$  Calc_Fitness(current_individual, cost, duration)
22:      if current_best < best_fitness then
23:        best_fitness  $\leftarrow$  current_best
24:        best_individual  $\leftarrow$  current_individual
25:    else
```

```

26:             break
27:         end if
28:     end if
29: end for
30: return best_individual
31: end function

```

5.2 Simulated Annealing

Introduction: Simulated Annealing is a metaheuristic optimization algorithm inspired by the process of annealing in metallurgy, where a material is heated and then slowly cooled to remove defects and reach a more stable structure.

Main Idea: Simulated Annealing balances exploration (accepting bad moves early on) and exploitation (refining good solutions later) by controlling a **temperature** parameter over time. In optimization, SA searches for a good solution by:

- Starting from a random solution
- Making small random changes (called perturbations)
- Accepting changes that improve the solution
- Occasionally accepting worse solutions (to escape local optima)
- Gradually reducing the chance of accepting worse solutions (by cooling)

Since SA and GA have a lot in common (**Perturbation/Get_Neighbor** and **Mutation, Decode, Calculate Fitness Score**), I won't be including any similar code here. But in the main loop introduced later on, there will be a few name changing, but the function still remains. This section will focus mainly on the differences that SA has as an algorithm.

5.2.1 State Representation

In Simulated Annealing, a state (also called a solution or configuration) represents one possible solution to the optimization problem. Unlike Genetic Algorithms (which work with a population of chromosomes), SA works with just one state at a time and gradually improves it.

The state representation of SA is similar with chromosome of GA.

Structure: State: {
 "task_order" : [3, 1, 4, 2, ..., 5],
 "team_assignment" : [2, 1, 3, 1, ..., 2]
}

5.2.2 Create Initial Feasible Solution

The initial solution (or initial state) is where Simulated Annealing starts its search. A good initial solution doesn't need to be optimal — it just needs to be:

- Feasible (respects problem constraints like precedence and availability)

- Complete (assigns all tasks with valid order and teams)

In this report, I will be giving two initial solution generate functions, one returns a greedy initial, another just returns a random feasible solution.

Pseudo Code

Operation: Generate Initial Solution

```

1: function GREEDY_CONSTRUCT(num_task, precedence, graph,
   team_available_time, available_team, task_duration, cost)
2:   in_deg  $\leftarrow$  [0] * num_task
3:   for (u, v) in precedence do
4:     in_deg[v]  $\leftarrow$  in_deg[v] + 1
5:   end for
6:   line  $\leftarrow$  queue()
7:   for task  $\leftarrow$  1 to num_task do
8:     if in_deg[task] == 0 then
9:       line push task
10:    end if
11:  end for
12:  order  $\leftarrow$  []
13:  while line is not empty do
14:    u  $\leftarrow$  pop line
15:    order append u
16:    for v in graph[u] do
17:      in_deg[v]  $\leftarrow$  in_deg[v] - 1
18:      if in_deg[v] == 0 then
19:        line push v
20:      end if
21:    end for
22:  end while
23:  taskStart  $\leftarrow$  [None] * num_task
24:  taskEnd  $\leftarrow$  [0] * num_task
25:  teamAvailable  $\leftarrow$  team_available_time
26:  task_order  $\leftarrow$  []
27:  team_assignment  $\leftarrow$  []
28:  for currentTask in order do
29:    bestChoice  $\leftarrow$  None
30:    bestTime  $\leftarrow$   $\infty$ 
31:    bestCost  $\leftarrow$   $\infty$ 
32:    for team in available_team[currentTask] do
33:      earliest  $\leftarrow$  max(teamAvailable[team], max(taskEnd[predecessors of
        currentTask]))
34:      currentCost  $\leftarrow$  cost(currentTask, team)
35:      if (earliest < best_time) or (earliest == best_time and currentCost <
        best_cost then
36:        best_choice, best_time, best_cost  $\leftarrow$  team, earliest, currentCost
37:      end if
38:    end for

```

```

39:     if best_choice  $\neq$  None then
40:         task_order append currentTask
41:         team_assignment append team
42:         taskStart[currentTask]  $\leftarrow$  best_time
43:         taskEnd[currentTask]  $\leftarrow$  best_time + duration(currentTask)
44:         teamAvailable[team]  $\leftarrow$  best_time + duration(currentTask)
45:     end if
46: end for
47: return {"task_order": task_order, "team_assignment": team_assignment}
48: end function

```

Helper Function: Random Topological Sort

```

1: function RANDOM_TOPOLOGICAL_SORT(num_task, precedence, can_do)
2:   in_deg  $\leftarrow$  [0] * num_task
3:   precedence_graph  $\leftarrow$  {u: [] for u in range(num_task)}
4:   for (u, v) in precedence do
5:     precedence_graph[u] append v
6:     in_deg[v]  $\leftarrow$  in_deg[v] + 1
7:   end for
8:   line  $\leftarrow$  queue([u for u in range(num_task) if in_deg[u] == 0])
9:   order  $\leftarrow$  []
10:  while line is not empty do
11:    shuffle line
12:    u  $\leftarrow$  pop line
13:    order append u
14:    for v in precedence_graph[u] do
15:      in_deg[v]  $\leftarrow$  in_deg[v] - 1
16:      if in_deg[v] == 0 then
17:        line push v
18:      end if
19:    end for
20:  end while
21:  return order
22: end function

```

Operation: Construct State

```

1: function RANDOM_CONSTRUCT(num_task, precedence, available_team, can_do)
2:   task_order  $\leftarrow$  Random_Topological_Sort(num_task, precedence, can_do)
3:   team_assignment  $\leftarrow$  [random_choice(available_team[i]) for i in task_order]
4:   return {"task_order": task_order, "team_assignment": team_assignment}
5: end function

```

5.2.3 Decode State and Fitness Function

The same **lexicographic fitness** is used:
fitness = (

```

    num_unscheduled_task,
    completion_time,
    total_cost
)

```

This is due to the similarity between *State* and *chromosome* of GA. This also leads to the **decode** and **calculate fitness** operations of SA is also similar to GA.

5.2.4 Neighbor Generation

The perturbation strategy in SA is structurally similar to the mutation operator used in GA. It involves randomly swapping tasks (while maintaining precedence) or reassigning teams. However, unlike GA, SA applies this change to a single current solution and uses a probabilistic acceptance criterion to decide whether to move to the new state, which is featured in the next section.

5.2.5 Acceptance Probability

In Simulated Annealing, a new solution is accepted if it's better than the current one. If it's worse, it might still be accepted with a probability:

$$P = \exp\left(-\frac{\Delta E}{T}\right)$$

Where:

- ΔE : Increased in cost (how much worse the new solution is)
- T : Current temperature

This allows the algorithm to escape *local minima* early on, then with the chance of accepting worse solutions gradually decrease as the temperature cool, solutions slowly stabilize.

Pseudo Code

Operation: Metropolis Criterion

```

1: function ACCEPTANCE(current_fitness, neighbor_fitness, temperature)
2:   if neighbor_fitness < current_fitness then
3:     return True
4:   end if
5:   delta ← sum(current_fitness - neighbor_fitness)
6:   prob ←  $\exp\left(\frac{\text{delta}}{\text{temperature}}\right)$ 
7:   r ← random number between 0 and 1
8:   return r ≤ prob
9: end function

```

5.2.6 Termination Condition

In SA, the criteria of termination is exactly similar to that of GA.

5.2.7 Cooling Schedule and Main Loop

After each iteration, temperature is decreased by a low amount, decreasing the probability of accepting worse solution, slowly increase the steadiness of solutions. Combining all the above operations, we get the main loop.

Pseudo Code

Operation: Main Loop

```
1: function MAIN
2:   start  $\leftarrow$  time.time()
3:   max_iter  $\leftarrow$  10000
4:   max_time  $\leftarrow$  300
5:   max_converge  $\leftarrow$  100
6:   Temp, alpha, T_min  $\leftarrow$  100, 0.95, 1e-2
7:   curr  $\leftarrow$  Greedy_Construct(num.task, precedence, graph, team.available_time,
   available_team, task_duration, cost)
8:   best  $\leftarrow$  curr
9:   f_best  $\leftarrow$  Calc_Fitness(best, cost, duration)
10:  for i  $\leftarrow$  1 to max_iter do
11:    if time.time() - start > max_time then
12:      break
13:    end if
14:    neighbor  $\leftarrow$  Get_Neighbor(curr, precedence, available_team, 5, 0.05)
15:    f_curr  $\leftarrow$  Calc_Fitness(curr, cost, duration)
16:    f_neig  $\leftarrow$  Calc_Fitness(neighbor, cost, duration)
17:    if Acceptance(f_curr, f_neig, Temp) then
18:      current  $\leftarrow$  neighbor
19:      if f_neig < f_best then
20:        best  $\leftarrow$  neighbor
21:        f_best  $\leftarrow$  f_neig
22:      end if
23:    end if
24:    Temp  $\leftarrow$  Temp * alpha
25:    if Temp < T_min then
26:      break
27:    end if
28:  end for
29:  return best
30: end function
```

5.3 Tabu Search

Introduction: Similar to Simulated Annealing, Tabu Search is also a meta-heuristic optimization method improved from hill climbing. Tabu Search introduces a memory-based local optimum escaping method.

Main Idea: Tabu Search inherits the greedy of Hill Climbing by always choosing the best neighbor from a neighbor list. But the robust lies at the tabu list.

- The *tabu list* stores recent moves (or solution attributes) that are considered "tabu" - temporary forbidden - to avoid immediately undoing recent changes
- The length of time a move stays tabu is called the *tabu tenure* (usually measures in iterations)

This helps the algorithm:

- Avoid cycling back to recent solutions
- Escape local optimum
- Maintain diversity in the search

5.3.1 State and Move Representation

This is exactly similar to "State Representation" of SA. A solution of TS is defined by 2 part: *task order* and *team assignment* **Structure: State:** {

```
"task_order" : [3, 1, 4, 2, ..., 5],
"team_assignment" : [2, 1, 3, 1, ..., 2]
```

} But beside from State, TS also introduce a concept called *Move*. Moves are saved in the *Tabu List* instead of saving an entire state in order to reduce memory usage. A *Move* is defined as follows:

```
Move: (
    "composite",,
    ("swap", 4, 2),
    [("assign", 4, 2, 3), ("assign", 5, 6, 2)]
)
OR ("noop",)
```

A move can be either "composite" or "noop". "Composite" saying that the move did changed something in the solution, either swapping two task start time or assigning new team to a task. The action "swap" indicates the swapping in task order, in this case, the task 4 is swapped with the task 2. The action "assign" indicates the re-assigning of tasks, in this case, task 4, was assigned to team 2, now re-assigned to team 3, and task 5 is reassigned to team 2 from team 6. And "noop" just says not changing.

5.3.2 Create Initial Feasible State

Similar with SA, the Initial State is also where TS start exploring, and the initialization of TS is exactly same with SA.

5.3.3 Decode State and Fitness Calculation

Again, similar with SA.

5.3.4 Neighbor Generation

The process of Neighbor Generation is similar to this of SA, but instead of just generating one neighbor, TS generate multiple, and greedily select the best one out of them if it is not violating the tabu list.

Pseudo Code

Operation: Neighbors Generate

```
1: function NEIGHBORS_GENERATE(curr, precedence, attempts, mutation_rate,
   num_neighbors)
2:   neighbors  $\leftarrow$  []
3:   for i  $\leftarrow$  1 to num_neighbors do
4:     neighbor  $\leftarrow$  Get_Neighbor(curr, precedence, attempts, mutation_rate)
5:     neighbors append neighbor
6:   end for
7:   return neighbors
8: end function
```

5.3.5 Move Identification and Tabu Check

This is where TS makes the difference. The helper function `Identify_Move` is used in order to classify neighbors, preventing cycling back to old moves.//

Pseudo Code

Helper Function: Identify Move

```
1: function IDENTIFY_MOVE(old_state, new_state)
2:   old_order, old_team  $\leftarrow$  old_state
3:   new_order, new_team  $\leftarrow$  new_state
4:   order_change, team_changes  $\leftarrow$  None, []
5:   for (a, b) in (old_state, new_state) do
6:     if a  $\neq$  b then
7:       order_change  $\leftarrow$  ("swap", a, b)
8:       break
9:     end if
10:  end for
11:  for i, (a, b) in ([1 to length(old_team)], old_team, new_team) do
12:    if a  $\neq$  b then
13:      team_changes append ("assign", new_order[i], a, b)
14:    end if
15:  end for
16:  if order_change or team_changes then
17:    return ("composite", order_change, team_changes)
18:  end if
19:  return ("noop",)
20: end function
```

Operation: Tabu Check

```
1: function TABU_CHECK(move, tabu_list)
2:   if move[0] is "noop" then
3:     return false
4:   end if
5:   swap, reassigns  $\leftarrow$  move[1], move[2]
```

```

6:   swap_list, assign_list ← tabu_list["task_swap"], tabu_list["team_reassign"]
7:   if swap in swap_list OR any([reassign in assign_list for reassign in reassigns])
   then
8:       return false
9:   end if
10:  return true
11: end function

```

5.3.6 Termination Condition

Termination condition of TS is similar to SA or GA termination condition.

5.3.7 Main Loop

Applying all the above operations, we have our main loop:

Pseudo Code

```

                                Main Loop

1: function MAIN
2:   start ← time.time()
3:   max_iter ← 10000
4:   max_time ← 300
5:   max_convergence ← 100
6:   tabu_list ← {"task_swap": , "team_reassign": }
7:   tabu_tenure ← 10
8:   curr ← Greedy_Construct(num_task, precedence, graph, team_available_time,
   available_team, task_duration, cost)
9:   best ← curr
10:  f_best ← Calc_Fitness(best, cost, duration)
11:  for i ← 1 to max_iter do
12:      if time.time() - start > max_time then
13:          break
14:      end if
15:      neighbors ← Get_Neighbor(curr, precedence, available_team, 5, 0.05)
16:      best_neighbor ← None
17:      best_neighbor_fit ← ∞
18:      for neighbor in neighbors do
19:          move ← Identify_Move(curr, neighbor)
20:          fit ← Calc_Fitness(neighbor, cost, duration)
21:          if Tabu_Check(move, tabu_list) OR fit < f_best then
22:              if fit < best_neighbor_fit then
23:                  best_neighbor ← neighbor
24:                  best_neighbor_fit ← fit
25:                  selected_move ← move
26:              end if
27:          end if
28:      end for
29:      if best_neighbor is None then

```

```

30:         break
31:     end if
32:     curr  $\leftarrow$  best_neighbor
33:     if best_neighbor_fit < best_fit then
34:         best  $\leftarrow$  best_neighbor
35:         best_fit  $\leftarrow$  best_neighbor_fit
36:     end if
37:     tabu_list["task_swap"][selected_move[1]]  $\leftarrow$  i + tabu_tenure
38:     for assign in selected_move[2] do
39:         tabu_list["team_reassign"][assign]  $\leftarrow$  i + tabu_tenure
40:     end for
41:     expired_swap  $\leftarrow$  [m for m, expire in tabu_list["task_swap"].items() if ex-
    pire  $\leq$  i]
42:     expired_assign  $\leftarrow$  [m for m, expire in tabu_list["team_reassign"].items() if
    expire  $\leq$  i]
43:     for (swap, assign) in (expired_swap, expired_assign) do
44:         del tabu_list["task_swap"][swap], tabu_list["team_reassign"][assign]
45:     end for
46: end for
47: return best
48: end function

```

Chapter 6

Results and Conclusion

6.1 Results

Below is the final result of each and every algorithms introduced in this report (from exact method to meta-heuristic) using given test cases. The results will be presented as a tuple of 3 integers indicating 3 consecutive objectives:

- Number of tasks assigned
- Total completion time
- Total assigning cost

For consistency and fairness, all algorithms random seed will be 42 and their termination condition will be as follows:

1. **Maximum number of Generation/Iterations:** 10000
2. **Maximum Time Limit:** 500 seconds

Input Size			Contraints Programming		Linear Programming		Backtracking	
Tasks	Teams	Precedence	Score	Runtime	Score	Runtime	Score	Runtime
5	6	4	(5, 235, 145)	2.074	(5, 235, 175)	0.038	(5, 235, 145)	0.016
10	15	6	(10, 190, 480)	2.313	(10, 230, 420)	0.376	NO OUTPUT	N/A
100	80	100	(10, 550, 1850)	15.984	NO OUTPUT	N/A	NO OUTPUT	N/A
200	80	150	(200, 350, 5740)	55.436	NO OUTPUT	N/A	NO OUTPUT	N/A
500	200	650	NO OUTPUT	N/A	NO OUTPUT	N/A	NO OUTPUT	N/A
600	300	750	NO OUTPUT	N/A	NO OUTPUT	N/A	NO OUTPUT	N/A
700	300	750	NO OUTPUT	N/A	NO OUTPUT	N/A	NO OUTPUT	N/A
800	400	750	NO OUTPUT	N/A	NO OUTPUT	N/A	NO OUTPUT	N/A
900	400	750	NO OUTPUT	N/A	NO OUTPUT	N/A	NO OUTPUT	N/A
1000	400	1750	NO OUTPUT	N/A	NO OUTPUT	N/A	NO OUTPUT	N/A

Figure 6.1: Results of Exact Methods.

Input Size			Greedy Algorithm		Simple Hill Climbing		Steepest-Ascent Hill Climbing	
Tasks	Teams	Precedence	Score	Runtime	Score	Runtime	Score	Runtime
5	6	4	(5, 235, 145)	0.000	(5, 235, 145)	0.305	(5, 235, 145)	1.997
10	15	6	(10, 210, 430)	0.000	(10, 190, 560)	0.499	(10, 190, 500)	2.816
100	80	100	(100, 550, 4510)	0.016	(100, 550, 4510)	11.870	(100, 550, 4510)	67.767
200	80	150	(200, 370, 13240)	0.024	(200, 370, 13240)	38.617	(200, 370, 13240)	220.879
500	200	650	(493, 840, 38150)	0.084	(493, 780, 48950)	344.263	(493, 780, 48280)	499.117
600	300	750	(599, 550, 42300)	0.188	(599, 550, 42300)	499.045	(599, 550, 42300)	499.067
700	300	750	(657, 510, 46270)	0.206	(657, 510, 46270)	499.040	(657, 510, 46270)	499.001
800	400	750	(786, 530, 63960)	0.187	(786, 520, 80350)	499.059	(786, 530, 63960)	499.233
900	400	750	(884, 820, 74300)	0.266	(884, 710, 91140)	499.335	(884, 710, 93130)	499.145
1000	400	1750	(980, 770, 68520)	0.340	(980, 770, 68520)	499.067	(980, 770, 68520)	499.677

Figure 6.2: Results of Heuristic Methods.

Input Size			Genetic Algorithm		Simulated Annealing		Tabu Search	
Tasks	Teams	Precedence	Score	Runtime	Score	Runtime	Score	Runtime
5	6	4	(5, 235, 145)	0.107	(5, 235, 145)	0.146	(5, 235, 145)	0.815
10	15	6	(10, 190, 480)	4.120	(10, 190, 570)	0.561	(10, 190, 490)	1.701
100	80	100	(100, 550, 4510)	122.468	(100, 550, 4510)	15.392	(100, 550, 4510)	12.270
200	80	150	(200, 370, 13240)	446.383	(200, 370, 13240)	42.164	(200, 370, 13240)	30.197
500	200	650	(493, 780, 49350)	499.125	(493, 780, 48760)	360.201	(493, 780, 46740)	202.745
600	300	750	(599, 550, 42300)	499.157	(599, 550, 42300)	499.131	(599, 550, 42300)	295.827
700	300	750	(657, 510, 46270)	499.358	(657, 510, 46270)	499.037	(657, 510, 46270)	396.692
800	400	750	(786, 530, 63960)	499.238	(786, 530, 63960)	499.019	(786, 530, 63960)	499.033
900	400	750	(884, 710, 92110)	499.361	(884, 730, 95380)	499.070	(884, 710, 88840)	499.011
1000	400	1750	(980, 770, 68520)	499.953	(980, 770, 68520)	499.071	(980, 770, 68520)	499.533

Figure 6.3: Results of Meta-Heuristic Methods.

6.2 Conclusion

Scalability

- **Exact Methods** are **not** scalable for large input sizes.
- **Heuristic** and **Meta-Heuristic** can handle much larger inputs, though computational requirement may varies among the algorithms.

Performance

- At small inputs, **Exact Methods** dominant.
- **Heuristic Methods** do give a reasonable result at an extremely low runtime, especially Greedy Algorithm.
- **Meta-Heuristic Methods** trade runtime to get high quality answer, close to exact.

Recommended Approaches

- For small inputs, **Exact Methods** is the best choice.
- For moderate-large inputs, **Meta-Heuristic Methods** offer a good balance between quality and scalability.
- Choose **Heuristic Methods** when time is a critical constraint and suboptimal are acceptable.
- Using a **Hybrid Approach**, meaning the combination of **Greedy Initialize** and **Meta-Heuristic** improvement, is recommended in any scenarios.